

Лабораторная работа №7

Отладка проекта

1 Цель работы

- 1.1 Изучить процесс отладки приложений,
- 1.2 Изучить процесс применения логирования проектов на C# с целью отладки.

2 Литература

2.1 Гагарина, Л. Г. Технология разработки программного обеспечения : учебное пособие / Л. Г. Гагарина, Е. В. Кокорева, Б. Д. Сидорова-Виснадул ; под редакцией Л. Г. Гагариной. – Москва : ИД «ФОРУМ» : ИНФРА-М, 2023. – 400 с. – URL: <https://znanium.com/read?id=416426>.– Режим доступа: для зарегистрир. пользователей. – Текст : электронный. – гл.5.

2.2 <https://habr.com/ru/articles/354962/>

3 Подготовка к работе

- 3.1 Повторить теоретический материал (см.п.2).
- 3.2 Изучить описание лабораторной работы.

4 Основное оборудование

- 4.1 Персональный компьютер.

5 Задание

5.1 Логирование исключений в файл

1. Создать консольное приложение, которое запрашивает ввод двух целых чисел и выполняет сложение, вычитание, умножение и деление с выводом результата.

2. Обернуть код в блок try-catch, который будет перехватывать исключения разных типов (например, FormatException, OverflowException и т.д.), каждый тип исключения – в своем блоке catch, включая тип Exception как fallback.

3. При возникновении исключения вывести в консоль сообщение об ошибке и текст исключения.

4. Добавить логирование исключений в текстовый файл log.txt, где будет сохраняться информация об исключениях (время возникновения, тип исключения и сообщение и весь текст исключения ex.ToString()). Для записи в файл можно использовать File.AppendAllText.

Образец записи:

[2026-09-10 14:23:11] FormatException: Input string was not in a correct format

...

5. Проверить, что логирование срабатывает, если пользователь вводит некорректные данные, например, текст вместо числа или числа, которые в результате дадут переполнение.

5.2 Применение логирования для отслеживания времени выполнения операций

Stopwatch – инструмент для измерения времени выполнения операций, особенно полезный в отладке производительности:

Stopwatch stopWatch = new();

```
stopWatch.Start();  
... // длительная операция  
stopWatch.Stop();  
TimeSpan ts = stopWatch.Elapsed; // получение прошедшего времени
```

1. Создать программу, которая выполняет несколько длительных операций, например: чтение данных из файла, получение данных по API, выполнение сложных математических расчетов.

2. Перед началом каждой операции запустить Stopwatch, а после завершения – остановить его. Выполнить каждую операцию 3 раза и вычислить среднее время по каждой операции.

3. Использовать Debug.WriteLine (для вывода в консоль отладки) и File.AppendAllText (для записи в лог timings.log) времени, потраченного на каждую операцию.

Образец записи:

```
[2026-09-10 14:23:11] Operation=ReadTxtFile, Elapsed=123 ms
```

4. Вывести в конце общее время выполнения операций.

5. Проанализировать лог и сделать выводы, какие операции выполняются дольше других и почему.

5.3 Применение TraceSource и TraceListener для сохранения информации в файл

TraceSource даёт именованный источник трассировки, собственный набор слушателей (TraceListener) и управление уровнем детализации через SourceSwitch. Это позволяет разделять логи разных подсистем приложения и маршрутизировать их независимо.

```
// создание именованного источника трассировки  
var ts = new TraceSource("MyApp")  
{  
    Switch = new SourceSwitch("MyAppSwitch") { Level = SourceLevels.Information }  
};
```

```
// настройка слушателей этого источника  
ts.Listeners.Clear(); // удаление DefaultTraceListener, чтобы не дублировать в Output  
ts.Listeners.Add(new TextWriterTraceListener("trace.log", "fileListener"));  
ts.Listeners.Add(new ConsoleTraceListener()); // для вывода сообщений в консоль  
ts.AutoFlush = true; // строки попадают в файл сразу, а не при завершении процесса
```

```
// Запись сообщений разных уровней  
ts.TraceInformation("информация");  
ts.TraceEvent(TraceEventType.Warning, 1, "предупреждение");  
ts.TraceEvent(TraceEventType.Error, 2, "ошибка");
```

```
ts.Flush(); // сброс выходного буфера  
ts.Close(); // закрытие слушателей
```

1. Создать консольное приложение, которое запрашивает ввод двух целых чисел и выполняет сложение, вычитание, умножение и деление с выводом результата.
2. Создать именованный `TraceSource` (например, "Calculator") и настроить его `SourceSwitch` на уровень `SourceLevels.Information`. Добавить в `ts.Listeners`:
 - `TextWriterTraceListener` для записи всех сообщений трассировки в файл `trace.log`,
 - `ConsoleTraceListener` для дублирования сообщений в консоль.
3. Применить методы `ts.TraceInformation(...)`, `ts.TraceEvent(TraceEventType.Verbose, ...)`, `ts.TraceEvent(TraceEventType.Warning, ...)`, `ts.TraceEvent(TraceEventType.Error, ...)` для логирования операций в файл, включая:
 - данные о входных параметрах (уровень `Verbose`),
 - выполненные операции (уровень `Information`),
 - результаты (уровень `Information`),
 - ошибки (например, деление на ноль или переполнение) – уровень `Error`.
4. По завершении выполнения программы убедиться, что файл `trace.log` содержит все сообщения о выполнении операций.
5. Обратить внимание: сообщения уровня `Verbose` в лог не попадут, потому что `SourceSwitch.Level = SourceLevels.Information` их отсекает. Проверить это, изменив уровень на `SourceLevels.Verbose` и запустив приложение повторно с сохранением в файл `trace_verbose.log`.
6. Закрыть логирование после завершения работы, используя `ts.Flush()` и `ts.Close()`. Убедиться, что после `Close()` повторная запись в `ts` теряется – это ожидаемое поведение.

5.4 Использование `SourceSwitch` и `SourceLevels` для управления уровнем детализации логирования

`SourceSwitch` позволяет централизованно управлять тем, какие сообщения попадут в лог, а какие будут отсечены ещё до записи в `TraceListener`. Это критично для производительности: проверка уровня происходит до форматирования сообщения. Уровень задаётся в коде или через `appsettings.json` + `Microsoft.Extensions.Logging`.

```
// создание переключателя с уровнем детализации
var sourceSwitch = new SourceSwitch("MyAppSwitch")
{
    Level = SourceLevels.Warning
};
// привязка переключателя к источнику трассировки
var ts = new TraceSource("MyApp")
{
    Switch = sourceSwitch
};
```

`SourceLevels` – это битовая маска, которая определяет, какие `TraceEventType` пропускать. Чем выше уровень, тем больше категорий включается. Виды уровней:

All (все), Verbose, Information, Warning, Error, Critical, Off (ничего).

Например: уровень `Warning` включает `Warning` и всё, что выше по важности (`Error`, `Critical`). Уровень `Verbose` включает всё.

1. Создать консольное приложение, которое выполняет операции загрузки, сохранения и удаления данных (можно имитировать работу с хранилищем).

Добавить в него подробные сообщения трассировки разных уровней:

- `TraceEventType.Verbose` – детали входных данных;
- `TraceEventType.Information` – начало и завершение операций;
- `TraceEventType.Warning` – потенциальные проблемы (например, попытка удалить несуществующую запись);
- `TraceEventType.Error` – ошибки операций (ловить в блоках try-catch).

2. Создать `SourceSwitch` с именем "StorageSwitch" и привязать его к `TraceSource("Storage")`. Настроить `TextWriterTraceListener` для записи в файл `storage.log` и установить `AutoFlush = true`.

3. Настроить `SourceSwitch.Level = SourceLevels.Warning` так, чтобы при запуске логировались только сообщения уровня `Warning` и выше. Запустить приложение и зафиксировать, какие сообщения попали в `storage.log`.

4. Изменить уровень на `SourceLevels.Verbose` и запустить приложение повторно. Убедиться, что теперь логируются все сообщения, включая `Verbose` и `Information`. Сравнить количество строк в логе с предыдущим запуском.

5. Проверить поведение при уровне `SourceLevels.Off`: сообщения не должны записываться вообще. Это полезно для полностью отключённого логирования в production.

6 Порядок выполнения работы

6.1 Выполнить задание из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Как правильно организовать обработку исключений при вводе пользовательских данных и записывать информацию об ошибках в лог-файл?

8.2 Какие методы класса `Stopwatch` следует использовать для измерения времени выполнения операций?

8.3 Как настроить `Trace.Listeners` для записи отладочной информации в файл и какие данные следует включать в логи при выполнении операций?

8.4 Какие уровни детализации предоставляет `TraceSwitch` и как его использовать для гибкого управления логированием в зависимости от потребностей?

8.5 В чем преимущества использования `Trace.WriteLine` по сравнению с простой записью в файл?